

6. Perform basic video processing operations on the captured video

Aim:

To perform basic video processing operations by reading a captured video in Python and displaying it in slow and fast motion using the OpenCV library.

Procedure:

- Import the required libraries such as OpenCV (cv2) and NumPy.
- Read the input image using the cv2.imread() function.
- Create the required kernel/parameters for image processing.
- Apply the required image-processing operation using the appropriate OpenCV function.
- Display the original and processed images using cv2.imshow() and close the windows using cv2.destroyAllWindows().

Code:

```
import cv2
```

```
import numpy as np
```

```
img=cv2.imread(r"C:\Users\Maruthi\Downloads\METAL.jpg",0)
```

```
if img is None:
```

```
    print("Image not found")
```

```
    exit()
```

```
f=np.fft.fft2(img)
```

```
fshift=np.fft.fftshift(f)
```

```
rows,cols=img.shape
```

```
crow,ccol=rows//2,cols//2
```

```
mask=np.ones((rows,cols),dtype=np.uint8)
```

```
mask[crow-5:crow+5,ccol-50:ccol-40]=0
```

```
mask[crow-5:crow+5,ccol+40:ccol+50]=0
```

```
filtered=fshift*mask
```

```
ishift=np.fft.ifftshift(filtered)
```

```
img_back=np.fft.ifft2(ishift)
```

```
img_back=np.abs(img_back)
```

```
img_back=cv2.normalize(img_back,None,0,255,cv2.NORM_MINMAX)
```

```
img_back=np.uint8(img_back)
```

```
cv2.imshow("Original Image",img)
```

```
cv2.imshow("Filtered Image",img_back)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Sample Input

Original Video

Sample Output

Video in slow motion and fast motion

Result:

The captured video was successfully read using Python and displayed in slow-motion and fast-motion using OpenCV.